Image GPS Mapping

with Folium

Visualizing Photo Locations on an Interactive Map using Python

Python • ExifRead • Pandas • Folium • GPS • EXIF Metadata

Project Overview

Objective

Extract GPS coordinates from JPEG images and display them as interactive markers on a map

Input

Folder of JPEG images containing embedded GPS EXIF metadata (e.g. football court survey photos)

Output

An HTML file (map.html) with clickable markers showing exact photo locations

Algorithm & Workflow

1 **Start**

Define image folder path

2 **Scan Folder**

List all .jpg files with os.listdir()

3 **Read EXIF**

Open each file, extract GPS tags via exifread

4 **Convert DMS→DD**

Convert Degrees/Minutes/Seconds to decimal degrees

→ continues

5 **Store Data**

Append [filename, lat, lon] to list → DataFrame

6 **Build Map**

Create Folium map centred at mean coords

7 **Save HTML**

Add markers & save map.html

Libraries Used

os

File System Navigation

Built-in Python module for traversing directories and constructing file paths (os.listdir, os.path.join)

exifread

EXIF Metadata Extraction

Third-party library that reads Exchangeable Image File Format (EXIF) tags from JPEG images, including GPS coordinates

pandas

Data Management

Stores extracted GPS data in a DataFrame (tabular structure) for easy iteration and statistical operations like mean()

folium

Interactive Map Generation

Python wrapper around Leaflet.js; creates web-based interactive maps with markers, popups, and zoom controls, saved as HTML

Data Source & Extraction

📸 Where Does the Data Come From?

- JPEG images taken with GPS-enabled cameras or smartphones
- GPS coordinates are embedded as EXIF metadata inside the image file
- Source folder: D:\maping\images\footballcourt
- Tags read:
  - — GPS GPSLatitude
  - — GPS GPSLatitudeRef (N/S)
  - — GPS GPSLongitude
  - — GPS GPSLongitudeRef (E/W)

🔢 DMS → Decimal Degrees Conversion

GPS data is stored in Degrees, Minutes, Seconds (DMS) format. The formula:

$$DD = D + M/60 + S/3600$$

If Ref is 'S' or 'W' → multiply DD by −1

Example: 28°36'14"N
= 28 + 36/60 + 14/3600
= 28.6039°

Data Structures Used

List [ ]

data = [] — dynamic list that accumulates [filename, lat, lon] entries from each image

```
data.append(["img1.jpg", 28.603, 77.218])
```

→ Efficient for sequential appending during folder scan

Dictionary { }

tags — exifread returns a dict-like object mapping tag names to IFDTag objects

```
tags["GPS GPSLatitude"].values
```

→ Fast O(1) lookup of specific EXIF fields by key

DataFrame

pd.DataFrame(data, columns=["file","lat","lon"]) — stores all image records in tabular form

```
df.lat.mean() → map center latitude
```

→ Enables vectorised operations, easy iteration, and summary stats

Ratio Object

exifread IFD Ratio: each DMS component is a Ratio with .num and .den attributes

```
float(dms[0].num) / float(dms[0].den)
```

→ Preserves full numeric precision of GPS fractions

Modules & Code Functions

get_decimal_from_dms(dms, ref)

Pure conversion function. Receives a DMS tuple and hemisphere ref string. Returns a signed decimal float.

- Input : dms (list of Ratio), ref ('N','S','E','W')
- Output: float decimal degree
- Logic : DD = D + M/60 + S/3600; negate if S or W

process_images(folder)

Core data pipeline. Iterates over all .jpg files, reads EXIF, calls converter, and builds a pandas DataFrame.

- Input : folder path (string)
- Output: DataFrame [file, lat, lon]
- Handles: prints warning if GPS tags missing

create_map(df)

Visualisation function. Creates a Folium map centred on the data centroid, adds a marker per image, saves HTML.

- Input : DataFrame with lat/lon columns
- Output: map.html (interactive Leaflet map)
- Config: zoom_start=18 for street-level detail

Applications

🏟️ Sports Facility Mapping

Map infrastructure assets such as football courts, playgrounds, and sports arenas using drone or handheld photos

🌾 Agriculture & Field Survey

Track crop photos taken across large farms to monitor growth, disease, or irrigation at precise GPS locations

🏗️ Construction Monitoring

Document construction progress by geotagging site photos, enabling stakeholders to track work location by location

🚔 Crime Scene / Forensics

Map evidence photos at crime scenes for spatial analysis and court presentation

🌍 Environmental Research

Plot wildlife sightings, pollution samples, or environmental survey photos on a map for ecological studies

🏛️ Tourism & Heritage

Auto-generate photo maps of heritage sites, tourist spots, or travel diaries from geotagged camera photos

Project Input & Output

INPUT

File Type:	JPEG images (.jpg)
Source:	D:\maping\images\footballcourt
Requirement:	Images must have GPS EXIF tags embedded
Tags Needed:	GPS GPSLatitude, GPSLatitudeRef, GPS GPSLongitude, GPSLongitudeRef
Format:	DMS - Degrees, Minutes, Seconds (e.g. 28deg 36' 14" N)
Condition:	Files without GPS tags are skipped with a printed warning

>

OUTPUT

File Name:	map.html
Format:	Self-contained HTML file (no server required)
Map Engine:	Leaflet.js (via Folium) with OpenStreetMap tiles
Centre:	Auto-centred at mean latitude and mean longitude of all images
Zoom Level:	18 (street-level detail, ideal for facility surveys)
Markers:	One pin per image — clicking a marker shows the filename as popup

Summary

1. GPS coordinates are silently stored inside every JPEG taken by a smartphone or GPS camera as EXIF metadata.
2. exifread extracts these coordinates (in DMS format), which are converted to decimal degrees for standard geo calculations.
3. pandas provides a clean DataFrame structure to store, manage, and perform statistical operations on the GPS records.
4. folium wraps Leaflet.js to generate a fully interactive map saved as a self-contained HTML file — viewable in any browser.
5. The project is directly applicable to sports facility surveys, agricultural mapping, construction monitoring, and beyond.
6. The modular code design (two helper functions + main calls) makes it easy to extend for additional metadata or map styles.